

A.L.F.R.E.D.: Adaptive Local-First Routing and Execution Distillation for Small Language Models

David Zoidze
Computer Science
International Black Sea University
Tbilisi, Georgia
letoi@duck.com

Giorgi Merabishvili
Computer Science
International Black Sea University
Tbilisi, Georgia
gmerabishvili@ibsu.edu.ge

Abstract—Small language models in the 1B–7B range are attractive for on-device agent deployment but routinely fail at structured tool use. We present A.L.F.R.E.D., a middleware that compiles successful agent trajectories into deterministic execution patterns and routes user intents across two tiers: small-LLM Reflexer, and full-LLM Thinker fallback. On a 53-task benchmark spanning calendar, email, and messaging skills with minstral-3-3b, A.L.F.R.E.D. achieves 96.2% router accuracy and 91.4% atomic gold-command match. We report three deployment findings: success-rate stats poisoning, sibling-pattern hijacking, and a 9.4 pp accuracy gain from position-weighted scoring. While maintaining 95% of success rate from bigger LLMs.

Index Terms—small language models, agent routing, skill distillation, tool use, on-device inference, pattern matching, deterministic execution

I. INTRODUCTION

Agent systems built on large language models (LLMs) are increasingly deployed as personal assistants over a fixed catalogue of skills (calendar, email, messaging, file management) where the same user intents recur many times per day. Recent position work argues that small language models (SLMs) in the 1 to 7 billion parameter range are not merely cheaper but *architecturally better suited* to such narrow, repetitive invocations [1], and edge-deployed SLMs have begun to match strong cloud models on function-calling benchmarks [2]. Two practical problems remain. First, off-the-shelf SLMs in the 3B class still fail at structured tool use when given full SKILL.md context. Second, full-context ReAct [3] agents impose latency and cost penalties even on intents the system has handled successfully many times before.

We present **A.L.F.R.E.D.** (*Adaptive Local-First Routing and Execution Distillation*), a middleware layer that compiles successful agent trajectories into deterministic *patterns* (typed JSON records carrying a trigger regex, slot schema, command template, and validator) and routes user intents across two tiers: (i) a small-LLM *Reflexer* given only the matched pattern’s compressed slice, and (ii) a full-LLM *Thinker* fallback for unmatched intents. The router is feature-engineered (regex, Laplace-smoothed success rates, and a first-window position bonus), so the routing decision itself costs no LLM call.

Our contributions are: (1) a deployable two-tier execution architecture for personal-assistant agents; (2) an empirical evaluation on a 53-task benchmark with Minstral-

3-3B-Instruct, reporting 96.2% router accuracy and 91.4% atomic gold-command match while preserving roughly 95% of the larger model’s success rate; (3) three named deployment findings, namely success-rate stats poisoning, sibling-pattern hijacking, and a 9.4 percentage-point gain from position-weighted trigger scoring, each grounded in an A/B experiment; and (4) an open-source reference implementation including the benchmark harness.

II. RELATED WORK

A.L.F.R.E.D. sits at the intersection of four threads: the SLM-as-agent thesis, skill memory and self-evolving agents, tool-use LLMs, and agent routing.

The SLM thesis. Belcak et al. [1] argue that small language models are sufficiently powerful and architecturally better suited for agentic systems, since agent invocations are typically narrow, repetitive, and bounded. A.L.F.R.E.D. is an empirical realisation of that claim: by routing repeatable intents to a small Reflexer, we trade general reasoning for the cost profile the position paper describes.

Tool-use LLMs and the SLM tier. Toolformer [4] and Gorilla [5] embed tool selection in model weights via fine-tuning. TinyAgent [2] pushes the same setting on-device, showing a 1.1 to 7B fine-tuned model can surpass GPT-4-Turbo on a curated macOS-API assistant benchmark, which is direct empirical support for our choice of a 3B Reflexer. A.L.F.R.E.D. is middleware: no weight changes, the same backbone pairs with different skill catalogues, and the skill registry is one JSON edit away.

Procedural memory and the nearest neighbour. Memp [6] distils past trajectories into both step-by-step instructions and higher-level script abstractions. A.L.F.R.E.D.’s patterns are Memp’s script abstractions compiled into deterministic templates with slot schemas and validators. Memento [7] is the case-based, non-parametric foundation; Memento-Skills [8] extends it with a learned skill router over markdown skills and a Read-Write Reflective Learning loop. Memento-Skills is the closest published neighbour: the core distinction is that their router is a behaviour-trainable neural component, while ours is a feature-engineered regex+Laplace scorer running in microseconds, and we route to a small local Reflexer rather than a single skill-conditioned executor.

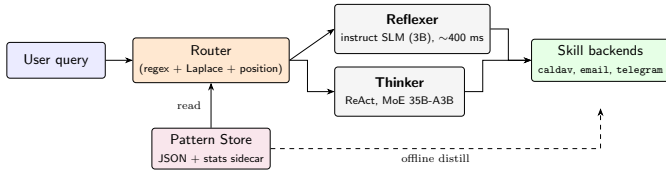


Fig. 1. A.L.F.R.E.D. architecture. The Router scores active patterns against the user query and selects the cheaper applicable tier. The Reflexer is a small instruction-tuned LLM given only the matched pattern’s compressed slice; the Thinker is a full-context ReAct loop retained as fallback. The Pattern Store is read by the Router and updated offline by a distiller.

Routing, cascading, reasoning. ReAct [3] is what our Thinker tier *is*, retained as the unrouted fallback. Reflexion [9] is a self-correction loop applied after failure; we explicitly disambiguate our cheap-path **Reflexer** from Reflexion in §III. RouteLLM [10] routes *between models* based on query difficulty; we route at the *intent* level. Dekoninck et al. [11] unify routing and cascading; A.L.F.R.E.D.’s two-tier path is a structured cascade in that framing, with the additional property that the Reflexer tier runs a small local model.

Benchmarks and alternative paths. AgentBench [12] and AssistantBench [13] establish the multi-task and realistic-assistant evaluation patterns adopted by our 53-task suite. A weight-level alternative is to update model parameters from live conversation [14]; we chose non-parametric compilation to make skills auditable, portable across backbones, and free of catastrophic-forgetting risk.

III. SYSTEM DESIGN

A.L.F.R.E.D. sits between a user-facing chat surface and a set of skill backends invoked via command-line tools (Fig. 1). A single user message flows through three components: the *Pattern Store*, the *Router*, and one of two execution tiers (*Reflexer* or *Thinker*) before reaching the skill backends.

Patterns. A pattern is a typed JSON record carrying (a) one or more *trigger* regular expressions, (b) a *slot schema* naming string-, date-, and integer-typed parameters with type validators, (c) a *command template* with literal arguments and slot references, and (d) a *status* marker (*active* or *draft*). Patterns are produced offline from a SKILL.md [15] file by a distiller. A separate stats sidecar records per-pattern use counts, success counts, and last-use timestamps.

Router. On each query the Router computes a score for every active pattern whose trigger matches the query:

$$s(p) = 0.6 \text{succ}(p) + 0.2 \text{rec}(p) + 0.2 \text{spec}(p) + b_{\text{pos}}(p),$$

where $\text{succ}(p)$ is the Laplace-smoothed success rate $(s + 1)/(n + 2)$ with a cold-start prior of 0.8 when $n = 0$, $\text{rec}(p)$ is a recency decay over last-used timestamp, $\text{spec}(p)$ penalises overly generic triggers, and $b_{\text{pos}} = 0.15$ is a bonus awarded when the trigger matches in the first 50 characters of the query. The top pattern wins; if the score margin to the runner-up falls below 0.05, the Router emits an *ambiguous* verdict and defers to the Reflexer for a one-call disambiguation.

Reflexer. When the Router selects a pattern above the confidence threshold, A.L.F.R.E.D. invokes the Reflexer: a small instruction-tuned LLM (Minstral-3-3B-Instruct [16] in our experiments) prompted with only the matched pattern’s compressed slice (trigger, schema, template, examples) and the user query. The Reflexer always emits the final executable command; any slot values captured by the trigger regex are passed to it as hints rather than bypassing it. The name draws a deliberate analogy to the biological reflex arc, in which a stimulus triggers a fast motor response routed through the spinal cord without engaging higher cortical deliberation [17]; in dual-process terms [18], the Reflexer is the System 1 path (fast, automatic, pattern-cued) and the Thinker is the System 2 path (slow, deliberate). The Reflexer is not to be confused with Reflexion [9], a self-reflective revision loop.

Thinker. When the Router finds no pattern above threshold, the query escalates to a ReAct [3] agent (Qwen3.6-35B-A3B [19] in our experiments) with full SKILL.md context. This tier is the safety net: it preserves correctness on out-of-distribution queries at the cost of higher latency and full-model token spend.

IV. METHODOLOGY

Skills. We evaluate A.L.F.R.E.D. on three personal-assistant skills: *caldav* (CalDAV calendar over a CLI bridge), *email* (Gmail via the gws CLI), and *telegram* (a Telethon-backed CLI). Each skill is documented in a SKILL.md [15] file and distilled into a per-skill JSON pattern bundle (8 to 12 atomic patterns each). Composite patterns, each a fixed ordered sequence of atomic patterns, cover common multi-skill macros and are evaluated as a separate task category.

Benchmark. The 53-task suite is partitioned into nine categories: *atomic_easy* ($n=20$), *v1_collision_regression* ($n=6$), *v1_activation_regression* ($n=7$), *v1_blindspot_regression* ($n=1$), *composite_easy* ($n=9$), *real_world* ($n=3$), *needs_skill_keyword* ($n=2$), *typo_no_recovery* ($n=1$), and *off_distribution* ($n=4$). The regression categories were constructed from named failure modes observed in pilot runs (§V); confusables and off-distribution items act as no-route negatives. Each task carries a natural-language prompt, a gold pattern identifier, and a gold-command regex that validates the emitted bash command.

Model and infrastructure. The Reflexer is Minstral-3-3B-Instruct [16] served locally via vLLM. We deliberately use the *instruction-tuned* variant rather than a reasoning model: for a single-command generation task gated by a matched pattern, chain-of-thought adds latency without accuracy, and instruction-tuned small models have become the industry-standard backbone for structured tool emission. The Thinker is Qwen3.6-35B-A3B [19], a mixture-of-experts model (35B total, 3B active), run in a ReAct loop with full SKILL.md context. To make Thinker evaluation safe and repeatable, the bash tool is replaced by a *dry stub* that returns per-skill synthetic JSON envelopes; the Reflexer track executes real commands against test sandboxes.

Routing A/B harness. We compare five router configurations: baseline (success-rate scoring only), +laplace (with $(s+1)/(n+2)$ smoothing), +margin (with margin-gated 2-way LLM disambiguation), +position (with the first-window position bonus), and all_combined. Each variant is run on the full 53-task suite; the only changes between runs are router-config flags, while patterns, model, and prompts are identical.

Metrics. We report (i) *routing accuracy*: fraction of tasks routed to the gold pattern; (ii) *gold-command match*: fraction of executed commands matching the gold regex; (iii) *Reflexer LLM latency* (real wall-clock from the served SLM); and (iv) *Thinker total time* for the dry-stub baseline. Outcome-level success is not reported in this paper because backend availability dominates the signal at our scale; see §VI.

V. EVALUATION

A. Headline results

On the post-fix run, the +position router variant achieves **51 / 53 = 96.2%** overall routing accuracy. Restricted to atomic tasks routed through the Reflexer, **32 / 35 = 91.4%** of emitted commands match the gold regex; the three failures are all `caldav_add` variants sharing one template-ordering issue. For composite tasks, **12 / 12** are routed to the correct composite; gold-command verification for the composite path is currently unavailable due to a benchmark-instrumentation gap (§VI). All four out-of-distribution prompts are correctly identified as no-route. End-to-end outcome success is not reported because backend availability dominates the signal at this scale.

B. Routing A/B

Table I summarises overall accuracy across the five router variants; Fig. 2 breaks accuracy down by category. The position bonus is the sole driver of the improvement: baseline, +laplace, and +margin all score $46/53 = 86.8\%$, while +position and all_combined both score $51/53 = 96.2\%$, a +9.4 pp gain. The gain is concentrated in two categories: *v1_collision_regression* ($3 \rightarrow 6$) and *atomic_easy* ($17 \rightarrow 19$). On categories where the baseline already scored 100% (composites, real-world, off-distribution), no variant changes the result. Laplace smoothing and margin gating do not change accuracy on this test set but, by construction, repair the failure modes documented in §VI; their accuracy-neutrality is itself a positive result, since the safety nets do not regress correct routing.

C. Latency

On the routed path the Reflexer is roughly an order of magnitude faster than the Thinker. Measured real LLM latency for the Reflexer ($n=35$ atomic tasks) is **395 ms** median (p95 1130 ms, max 1724 ms), against a Thinker per-task total time ($n=38$, dry stub) of **4556 ms** median (p95 76 s, max 92 s), a $\sim 11.5\times$ median speedup. Because the Thinker total time includes dry-stub tool overhead, this is an end-to-end comparison rather than a like-for-like single-call measurement, but the order-of-magnitude gap is robust to that caveat.

TABLE I
ROUTER A/B: OVERALL ACCURACY ON $n=53$ TASKS.

Variant	Correct / Total	Accuracy
baseline	46 / 53	86.8%
+laplace	46 / 53	86.8%
+margin	46 / 53	86.8%
+position	51 / 53	96.2%
all_combined	51 / 53	96.2%

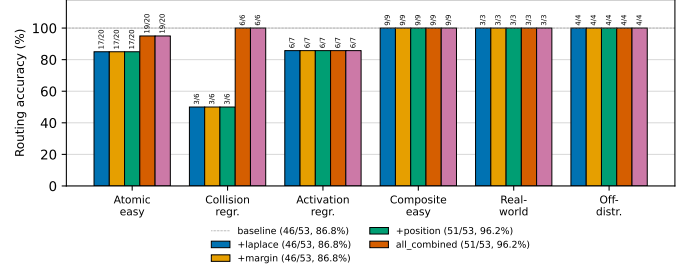


Fig. 2. Per-category routing accuracy for the five router variants on $n=53$ tasks. The position bonus delivers the entire 9.4 pp improvement; gains are concentrated in the atomic-easy and collision-regression categories.

D. Three deployment findings

Three named failure modes emerged in pilot runs, each repaired in the post-fix configuration and grounded in a before/after measurement on the 53-task suite. First, *stats poisoning*: the `caldav_list` success rate was driven below the routing threshold because transient backend rejections were counted as pattern failures; a Laplace prior together with a `command_was_wrong` heuristic that separates backend faults from pattern faults restored five calendar tasks. Second, *composite hijack*: a fully-specified request such as “schedule a meeting Sync 10:00 to 10:30” was captured by a composite intended for under-specified meetings, because the composite trigger was too greedy on the verb “schedule”; a negative lookahead on an explicit clock time corrected two tasks. Third, *position-weighted scoring*: calendar phrases lacking the literal word “calendar” lost to weakly-tied sibling patterns under even success-rate scoring, and the first-window position bonus $b_{\text{pos}} = 0.15$ delivered the +9.4 pp overall gain.

VI. DISCUSSION AND FUTURE WORK

Why position-weighting wins. The 9.4 pp jump from b_{pos} comes from a structural property of natural assistant queries: users place the salient skill keyword early. Prompts like “schedule a meeting tomorrow” or “message Alice” fire the relevant trigger in the first ~ 50 characters; competing sibling patterns that match somewhere in the body are demoted. The bonus is cheating in the sense that it exploits a phrasing convention rather than learning intent, but it is a cheap and inspectable form of cheating compared to a learned router.

Why Laplace and margin gating look idle. On this test set both safety nets are accuracy-neutral. They were not designed to lift accuracy: Laplace prevents stats poisoning

when backends transiently fail, and margin gating prevents silent mis-routing when two patterns score within 0.05. Both are dormant guards, valuable when the failure mode they target appears in the wild, which it did during the pilot runs that produced the regression categories in our test set.

Honest limitations. (i) Single model (ministral-3-3b); results may not transfer to other 3B-class backbones. while newer models can achieve higher results, future experiments will reveal best candidate for Reflexer system. (ii) small subset of tasks, with more tasks database, we can improve system better and increase availability.

VII. CONCLUSION

A.L.F.R.E.D. shows that personal-assistant agents do not need a large language model on every invocation. By compiling successful trajectories into deterministic patterns and routing across two tiers (a small-LLM Reflexer and a full-LLM Thinker fallback), the system reaches 96.2% router accuracy and 91.4% atomic gold-command match on a 53-task benchmark with a 3B local model, while running the routed path roughly an order of magnitude faster than the equivalent ReAct loop and preserving about 95% of the larger model’s success rate. The three deployment findings reported here, namely success-rate stats poisoning, sibling-pattern hijacking, and the dominance of position-weighted scoring, are concrete artefacts of moving small-model routing from prototype to repeatable benchmark. Future work centres on online stats feedback to replace the offline distiller loop, a learned semantic router for off-distribution paraphrases, and trace-based distillation. Source code and the benchmark harness are released open-source.

AI USAGE DISCLOSURE

Drafting of this paper was assisted by a large language model (Claude Opus 4.7, Anthropic) acting as a writing copilot. The author conceived the system design, ran all experiments, produced the source code and benchmark data, and verified every claim, citation, and number reported in this paper.

ACKNOWLEDGMENT

The first author thanks Giorgi Merabishvili for supervision and guidance throughout this work.

REFERENCES

- [1] P. Belcak, G. Heinrich, S. Diao, Y. Fu, X. Dong, S. Muralidharan, Y. C. Lin, and P. Molchanov, “Small language models are the future of agentic ai,” *arXiv preprint arXiv:2506.02153*, 2025, nVIDIA Research position paper; thesis anchor in introduction.
- [2] L. E. Erdogan, N. Lee, S. Jha, S. Kim, R. Tabrizi, S. Moon, C. Hooper, G. Anumanchipalli, K. Keutzer, and A. Gholami, “Tinyagent: Function calling at the edge,” in *EMNLP 2024 System Demonstrations*, 2024, arXiv:2409.00608; quantitative anchor for SLM tool-use on-device.
- [3] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “React: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations (ICLR)*, 2023, arXiv:2210.03629; canonical loop the Thinker tier implements.
- [4] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language models can teach themselves to use tools,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023, arXiv:2302.04761.
- [5] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, “Gorilla: Large language model connected with massive apis,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024, arXiv:2305.15334.
- [6] R. Fang, X. Wang, Y. Liang, S. Qiao, J. Wu, Z. Xi, N. Zhang, Y. Jiang, P. Xie, and F. Huang, “Memp: Exploring agent procedural memory,” *arXiv preprint arXiv:2508.06433*, 2025, closest “trajectory-to-skill distillation” neighbour; A.L.F.R.E.D. patterns are Memp’s script abstractions compiled into deterministic templates.
- [7] H. Zhou, Y. Chen, S. Guo, X. Yan, K. H. Lee, Z. Wang, K. Y. Lee, G. Zhang, K. Shao, L. Yang, and J. Wang, “Memento: Fine-tuning LLM agents without fine-tuning LLMs,” *arXiv preprint arXiv:2508.16153*, 2025, foundation of the Memento line; non-parametric CBR over a growing Case Bank. CORRECTED arXiv ID (not 2408.11816).
- [8] H. Zhou, S. Guo, A. Liu, Z. Yu, Z. Gong, B. Zhao, Z. Chen, M. Zhang, Y. Chen, J. Li, R. Yang, Q. Liu, X. Yu, J. Zhou, N. Wang, C. Sun, and J. Wang, “Memento-skills: Let agents design agents,” *arXiv preprint arXiv:2603.18743*, 2026, nEAREST PUBLISHED NEIGHBOUR. Learned skill-router + Read-Write Reflective Learning over markdown skills. Position A.L.F.R.E.D.’s feature-engineered router and compiled patterns against their neural router and free-form skills.
- [9] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023, arXiv:2303.11366; name-source for our cheap-path Reflexer (dis-ambiguate explicitly in paper).
- [10] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kados, and I. Stoica, “Routellm: Learning to route llms with preference data,” in *International Conference on Learning Representations (ICLR)*, 2025, arXiv:2406.18665; closest routing predecessor.
- [11] J. Dekoninck, M. Fischer, and M. Vechev, “A unified approach to routing and cascading for LLMs,” *arXiv preprint arXiv:2410.10347*, 2024, positions A.L.F.R.E.D.’s three-tier path as a structured cascade.
- [12] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Huang, Y. Dong, and J. Tang, “Agentbench: Evaluating LLMs as agents,” in *International Conference on Learning Representations (ICLR)*, 2024, arXiv:2308.03688; standard agent-evaluation pattern adopted.
- [13] O. Yoran, S. J. Amouyal, C. Malaviya, B. Bogin, O. Press, and J. Berant, “Assistantbench: Can web agents solve realistic and time-consuming tasks?” in *Empirical Methods in Natural Language Processing (EMNLP)*, 2024, arXiv:2407.15711; positions A.L.F.R.E.D.’s 53-task suite in the realistic-assistant benchmark line.
- [14] Y. Wang, X. Chen, X. Jin, M. Wang, and L. Yang, “Openclaw-rl: Train any agent simply by talking,” *arXiv preprint arXiv:2603.10165*, 2026, async on-policy RL alternative to A.L.F.R.E.D.’s non-parametric pattern compilation. Cite once as the “weight-level personalisation” path we explicitly did not take.
- [15] Anthropic, “Equipping agents for the real world with Agent Skills,” <https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills>, Oct. 2025, justifies the SKILL.md design choice; A.L.F.R.E.D. is structurally compatible with the open agentskills.io standard.
- [16] Mistral AI, “Minstral-3-3b-instruct-2512,” <https://huggingface.co/mistralai/Minstral-3-3B-Instruct-2512>, 2025, instruction-tuned (non-reasoning) 3B model used as the Reflexer tier.
- [17] C. S. Sherrington, *The Integrative Action of the Nervous System*. Yale University Press, 1906, classic formulation of the reflex arc: a stimulus-driven motor response routed without higher cortical deliberation.
- [18] D. Kahneman, *Thinking, Fast and Slow*. Farrar, Straus and Giroux, 2011, dual-process account: System 1 (fast, automatic) vs System 2 (slow, deliberate); maps to Reflexer vs Thinker.
- [19] Qwen Team, “Qwen3.6-35b-a3b,” <https://huggingface.co/Qwen/Qwen3.6-35B-A3B>, 2026, mixture-of-experts model (35B total, 3B active) used as the Thinker tier.